Instituicao de Ensino Demo

Curso: Engenharia de Software

2o Semestre

Material de Estudo

Estrutura de Dados

---

1. INTRODUCAO

Estrutura de Dados e uma das disciplinas mais importantes da computacao. Ela ensina como organizar e armazenar dados de forma eficiente para que operacoes como busca, insercao e remocao sejam rapidas. A escolha da estrutura de dados correta pode fazer a diferenca entre um programa que roda em milissegundos e um que leva horas. Todo engenheiro de software precisa dominar esse tema.

Ementa: Listas, pilhas e filas. Arvores binarias e de busca. Grafos. Tabelas hash. Algoritmos de ordenacao e busca. Complexidade de algoritmos (notacao Big-O). Recursividade.

2. CONCEITOS FUNDAMENTAIS

A seguir, apresentamos os conceitos-chave desta disciplina com explicacoes didaticas e exemplos praticos para facilitar o aprendizado.

2.1 Listas, Pilhas e Filas

Listas encadeadas armazenam elementos em nos conectados por ponteiros. Pilhas (LIFO - Last In, First Out) permitem insercao e remocao apenas no topo. Filas (FIFO - First In, First Out) inserem no final e removem do inicio. Cada estrutura tem aplicacoes especificas: pilhas para desfazer acoes, filas para processamento de tarefas.

Exemplo:

```
Pilha (LIFO) - exemplo com lista Python:
pilha = []
pilha.append("A") # push
pilha.append("B")
pilha.append("C")
topo = pilha.pop() # "C" (ultimo a entrar)

Fila (FIFO) - exemplo com deque:
from collections import deque
fila = deque()
fila.append("A") # enqueue
fila.append("B")
primeiro = fila.popleft() # "A" (primeiro a entrar)
```

2.2 Complexidade de Algoritmos (Big-O)

A notacao Big-O descreve o comportamento de um algoritmo conforme o tamanho da entrada cresce. O(1) = constante, O(log n) = logaritmico, O(n) = linear, O(n log n) = linearitmico, O(n^2) = quadratico. Entender complexidade permite escolher o algoritmo mais eficiente para cada situacao.

Exemplo:

```
Complexidades comuns:
O(1): acesso a elemento por indice em array
O(log n): busca binaria
O(n): busca linear em lista
O(n log n): mergesort, quicksort (caso medio)
O(n^2): bubble sort, selection sort

Para n=1.000.000:
O(n) = 1.000.000 operacoes
```

```
O(n^2) = 1.000.000.000.000 operacoes!
```

3. RESUMO DOS PRINCIPAIS TOPICOS

O conteudo programatico da disciplina esta organizado nas seguintes unidades:

- Unidade 1: Revisao de ponteiros e alocacao dinamica
- Unidade 2: Listas encadeadas (simples, dupla, circular)
- Unidade 3: Pilhas e filas - implementacao e aplicacoes
- Unidade 4: Arvores binarias e arvores de busca (BST)
- Unidade 5: Arvores balanceadas (AVL) e heaps
- Unidade 6: Tabelas hash - funcoes de dispersao e colisoes
- Unidade 7: Grafos - representacao, BFS, DFS
- Unidade 8: Algoritmos de ordenacao (quicksort, mergesort, heapsort)

Cada unidade aborda um aspecto fundamental da disciplina, construindo progressivamente o conhecimento necessario para dominar o tema. Recomenda-se estudar as unidades na ordem apresentada.

4. EXERCICIOS DE FIXACAO

Resolva os exercicios abaixo para verificar seu entendimento dos conceitos apresentados.

Questao 1 (Multipla Escolha)

Qual estrutura de dados segue o principio LIFO?

a) Fila
b) Lista
c) Pilha
d) Arvore

Questao 2 (Dissertativa)

Explique a diferenca entre uma arvore binaria de busca (BST) e uma arvore AVL.

Questao 3 (Multipla Escolha)

Qual e a complexidade media do quicksort?

a) O(n)
b) O(n log n)
c) O(n^2)
d) O(log n)

Questao 4 (Dissertativa)

Descreva como funciona uma tabela hash e como colisoes sao tratadas.

5. DICAS DE ESTUDO E RECURSOS ADICIONAIS

Para maximizar seu aprendizado nesta disciplina, siga estas recomendacoes:

- Implemente cada estrutura de dados do zero — nao apenas use bibliotecas prontas.
- Visualize as estruturas com desenhos e diagramas.
- Pratique analise de complexidade em todos os algoritmos que escrever.
- Resolva problemas no LeetCode focando em estruturas de dados.

Bibliografia Recomendada:

- CORMEN, T. et al. Algoritmos: Teoria e Pratica. Elsevier, 2012.
- ZIVIANI, N. Projeto de Algoritmos. Cengage Learning, 2010.
- GOODRICH, M.; TAMASSIA, R. Estruturas de Dados e Algoritmos em Java. Bookman, 2013.

6. GLOSSARIO DE TERMOS IMPORTANTES

Consulte este glossario sempre que encontrar um termo desconhecido durante seus estudos.

- **LIFO**: Last In, First Out — principio da pilha.
- **FIFO**: First In, First Out — principio da fila.
- **Big-O**: notacao que descreve a complexidade de tempo/espaco de um algoritmo.
- **Arvore binaria**: estrutura hierarquica onde cada no tem no maximo 2 filhos.
- **Hash**: funcao que mapeia dados de tamanho variavel para tamanho fixo.
- **Grafo**: estrutura com vertices (nos) e arestas (conexoes).
- **Recursividade**: tecnica onde uma funcao chama a si mesma.